

Authoring Standards-Conformant HAnim Humanoids with an X3D Model Context Protocol Server: A Human–AI Case Study

Alexander Hoffman

Claude (Anthropic Claude Code)

Draft — collaborative authorship

June 12, 2026

Abstract

We describe the collaborative construction of two interactive 3D demonstrations—a humanoid running a slalom course and a “living” classroom skeleton—built on the emerging ISO/IEC 19774 (HAnim) Level of Articulation 5 (LOA5) draft and delivered entirely through an X3D Model Context Protocol (MCP) server driven by a large language model. Rather than treating the model output as a finished artifact, we treat the human–AI pair as co-authors: the human supplies intent, domain pointers, and aesthetic judgment; the AI supplies generation, a tight verify-by-render feedback loop, and bug-level contributions back to the toolchain. We report the architecture (X3D MCP, the canonical `x3d.py` package, the `X3dToX3dom.xmlt` stylesheet, and the `X_ITE` web player), a chronological case study of the prompting that produced the demos, the practices that made it reliable, and concrete findings—including upstream bug reports and a roadmap for layering anatomy (clothing, ligaments, muscles, organs, skin) atop the skeleton.

1 Introduction

The popular framing “the AI built this” is both inaccurate and uninteresting. What actually produces good work is a *pairing*: a human who knows where the problem and the standards live, and a model that can generate, verify, and iterate at speed. This paper is a worked example. Over a single extended session, a human collaborator and an AI agent (Anthropic’s Claude, running in Claude Code) built two interactive X3D scenes featuring a fully articulated human skeleton conforming to the draft LOA5 level of the ISO/IEC 19774 Humanoid Animation (HAnim) standard, animated with the standard’s own walk/run/jump cycles, and viewable in a web browser.

The contribution is less the scenes themselves than the *method*: every artifact was produced through an X3D MCP server, validated against the X3D schema, and—crucially—visually verified by the agent rendering each scene and inspecting the resulting image before moving on. That loop, plus a decision to integrate the canonical standard rather than reinvent it, is what made the output trustworthy.

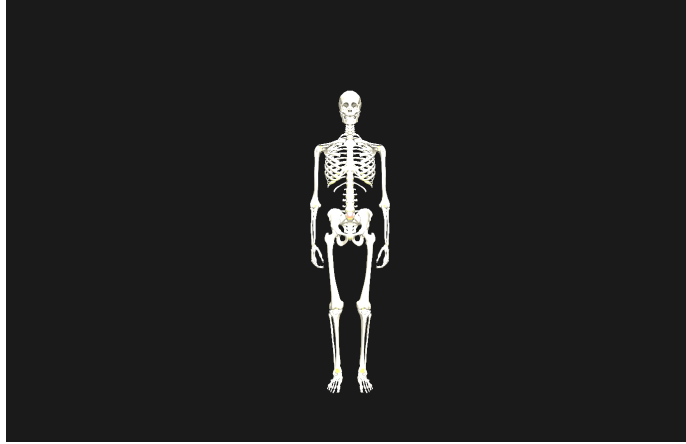


Figure 1: The canonical HAnim LOA5 bone-mesh humanoid (150 joints, 243 individual bone meshes from the Web3D AllBonesLOA5Skeletons model) rendered in Castle Model Viewer. This same figure is the actor in both demonstrations.

2 Background

X3D. Extensible 3D (X3D, ISO/IEC 19775) is the royalty-free ISO standard for declarative 3D scenes—the successor to VRML. A scene is a graph of *nodes* (geometry, transforms, lights, sensors, interpolators) connected by *ROUTES* that carry events. X3D 4.0 added a physically based rendering (PBR) material, `PhysicalMaterial`, and image-based lighting via `EnvironmentLight`.

HAnim and LOA. The Humanoid Animation standard (ISO/IEC 19774) defines a skeleton as a hierarchy of `HAnimJoint` nodes, each carrying a named `HAnimSegment` that holds geometry. A figure’s detail is graded by *Level of Articulation* (LOA): LOA-0 is a single root; the published 2019 standard tops out at LOA-4 (148 joints). The active draft adds **LOA5**, and the Web3D example archive ships an `AllBonesLOA5Skeletons` model—a 150-joint figure whose segments inline individual bone meshes—together with standardized Walk, Run, Jump and other motion cycles driven by interpolators.

The MCP. A Model Context Protocol server exposes tools an LLM can call. The X3D MCP server used here exposes scene construction, validation (against the X3D 4.x schema), semantic checks, encoding conversion, and X3DOM page rendering. The agent calls these tools the way a human would click an editor’s menus, but programmatically and at speed.

3 Setup: Tools, Installs, and Links

Everything below is free and open. Commands assume macOS with Homebrew (brew.sh); the equivalents on Linux/Windows are noted where they differ. This is the complete list—nothing else is required.

Core authoring and rendering.

- **Java (OpenJDK)** — runtime for Saxon and NetBeans. `brew install openjdk`. openjdk.org.

- **Apache NetBeans + X3D-Edit** — the standards X3D authoring tool (tree editor, schema/DTD validation, conversions). Install NetBeans (`brew install --cask netbeans`; netbeans.apache.org), then inside it: *Tools* → *Plugins* → *Available Plugins* → *search* “X3D-Edit” → *Install* (savage.nps.edu/X3D-Edit). On JDK 24+ add `-J-Djdk.xml.entityExpansionLimit=5000` to `netbeans.conf` so X3D’s large DOCTYPE does not trip the XML parser.
- **Castle Model Viewer** — GPU X3D viewer with command-line screenshots (`--screenshot`). castle-engine.io/castle-model-viewer (download the build for your OS; on macOS run `xattr -cr` on the app to clear the quarantine flag).

Generation and conversion (the canonical pipeline).

- **x3d.py** — the official Web3D Python package. `pip install x3d`. [web3d.org .../python](https://web3d.org/python).
- **Saxon-HE** — an XSLT 2.0 processor (the standard X3D stylesheets are XSLT 2.0, which the common `xsltproc/libxslt` cannot run). Fetch the self-contained jar:

```
curl -L -o saxon9he.jar \
https://repo1.maven.org/maven2/net/sf/saxon/Saxon-HE/9.9.1-8/Saxon-HE-9.9.1-8.jar
```

- **X3dToX3dom.xslt** — the official stylesheet that wraps an X3D scene into a web page for a chosen player. [web3d.org .../X3dToX3dom.xslt](https://web3d.org/X3dToX3dom.xslt).

Web players (no install; loaded from a CDN by the page).

- **X_ITE** — a WebGL X3D player with full HAnim support. create3000.github.io/x_ite (script via [jsDelivr](https://jsdelivr.com)).
- **X3DOM** — the other major WebGL X3D player. x3dom.org. (HAnim limitation noted in Section 7.)

The HAnim LOA5 assets (from the Web3D example archive). The skeleton and motion live at [web3d.org .../HumanoidAnimation/Bones/](https://web3d.org/HumanoidAnimation/Bones/): `AllBonesLOA5Skeletons.x3d` (the figure), `AllBonesLOA5SkeletonsInlineAnimation.x3d` (Walk/Run/Jump cycles), and the individual bone meshes the segments inline. The draft standard itself is at web3d.org/specifications/X3Dv4Draft/ISO-IEC19774.

Optional: texture generation and paper build.

- **mflux** (textures) — `pip install mflux` on Apple Silicon; weights download from [Hugging Face](https://huggingface.co) on first run. Non-Apple equivalent: FLUX.1 or Stable Diffusion via `diffusers` or ComfyUI, or any hosted image API (Section 4).
- **tectonic** (this paper) — a self-contained L^AT_EX engine that fetches packages on demand. `brew install tectonic`; tectonic-typesetting.github.io. Any T_EX distribution works too.
- **poppler** (`pdftoppm`, to preview the PDF as images) and a Chromium-based browser (for headless WebGL screenshots) are conveniences for the verify-by-render loop, not requirements.

4 Architecture and Toolchain

The pipeline separates *generation*, *authoring artifact*, and *delivery*:

1. **Generation** — **x3d.py**. The official Web3D Python package constructs X3D node objects programmatically and serializes valid XML. Building a 150-joint humanoid recursively in code is far more maintainable than string templating, and the objects validate by construction.
2. **Authoring artifact** — **the .x3d file**. The authoritative scene keeps real HAnim nodes, opens in X3D-Edit (the NetBeans-based authoring tool) and desktop players, and validates against the schema.
3. **Delivery** — **the web page**. The canonical `X3dToX3dom.xslt` stylesheet (run through the Saxon XSLT 2.0 processor) produces a browser page. For the player we use **X_ITE**, which has complete HAnim support and renders the real skeleton directly; X3DOM, by contrast, cannot yet render an HAnim skeleton (see Section 7).

Two supporting tools proved decisive. **Castle Model Viewer** (a free, cross-platform X3D/VRML/glTF viewer from the Castle Game Engine project) renders X3D on the GPU and—critically—can take a screenshot from the command line, which is how the agent “sees” its work (Section 6).

For textures we used **mflux**. To unpack the acronyms: **FLUX.1** is an open-weights text-to-image diffusion model (from Black Forest Labs); **MLX** is Apple’s array/machine-learning framework optimized for Apple Silicon GPUs; and **mflux** is a Python package that runs FLUX.1 on MLX. Together they generate an image from a text prompt entirely on a local Mac, with no cloud service. We used it to produce the classroom’s chalkboard and anatomy posters, applied as **ImageTextures**—closing a generate-image → place-in-3D loop. None of this is Apple-specific in principle: any text-to-image generator (e.g. FLUX.1 or Stable Diffusion via **diffusers**/ComfyUI on an NVIDIA GPU, or a hosted image API) substitutes directly; only the local runner changes.



Figure 2: A classroom chalkboard texture generated locally with mflux (FLUX.1-schnell) from a text prompt, then applied as an X3D **ImageTexture**. The image-generation and 3D-authoring loops compose.

5 Case Study: A Chronological Build

The demos were not specified up front; they emerged through iteration. The prompts below are simplified reconstructions that capture the *shape* of the collaboration.

5.1 From a stick figure to real HAnim

The first request was direct:

Let's get a human running animation. Use their LOA skeleton and the X3D MCP to make it run correctly, through a level.

An initial figure faked joints with nested `Transform` nodes. The decision to switch to *real* `HAnimJoint`/`HAnimSegment` nodes—using the standard joint names and centers from the Web3D archive—made the figure validate and open correctly in standards tools. A key simplification fell out: because an `HAnimJoint`'s rest pose is identity, all segment geometry can be authored in absolute coordinates and still articulate.

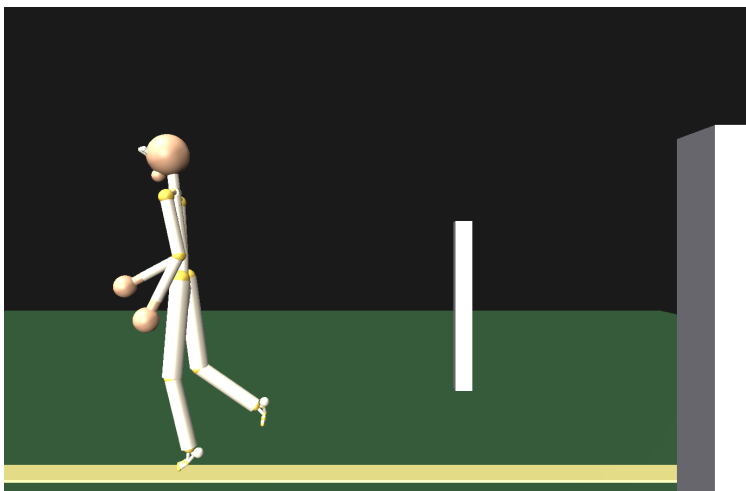


Figure 3: An early LOA-4 figure with generated stick geometry, running the slalom course. The gait cadence is synchronized to ground speed so the feet do not skate.

5.2 Adopting the canonical pipeline

A single pointed question redirected the whole approach:

Did you use the x3d.py export or the stylesheets to get to this?

The honest answer was “neither yet”—generation had been ad-hoc string templating and the web page a hand-rolled converter. *Lesson: a domain expert's question is a steering signal; the right response is to name the gap plainly, not paper over it.* The instruction followed:

Rebuild it on the canonical pipeline.

Generation moved to `x3d.py` and delivery to `X3dToX3dom.xslt` run through Saxon. This surfaced real serialization quirks (Section 7) but produced schema-valid, standards-aligned output that composes with the rest of the ecosystem.

5.3 Upgrading to LOA5 and the real bones

The collaborator located the draft LOA5 model and its standardized animation:

Use AllBonesLOA5 for both projects.

We downloaded the 243 individual bone meshes, extracted the canonical 150-joint humanoid as a reusable fragment, and *embedded* it so its `hanim_` joint DEFs were in scope for our animation ROUTEs. The same figure now stars in both demos.

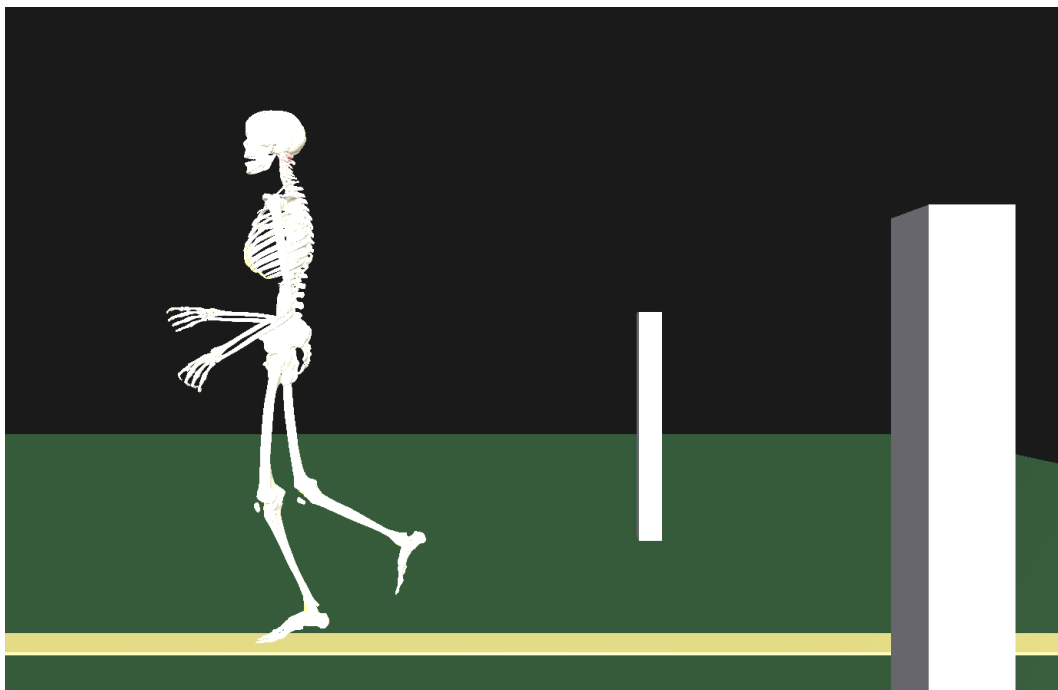


Figure 4: The canonical LOA5 bone-mesh skeleton running the slalom course, driven by our gait routed into the standard joints. Rendered in Castle Model Viewer.

5.4 Look development

The canonical meshes shipped with old-style Phong materials at very high ambient reflectance, which flattened all anatomical form under image-based lighting. A batch pass re-authored them as `PhysicalMaterial` (metallic 0, matte roughness), restoring depth:

Do the PBR-material pass for the rendered demo and document it. I'm not concerned with canon --- we are doing our demo. This is about what the MCP can do.

5.5 Interactivity

Finally, the classroom skeleton gained the standard's locomotion cycles plus clickable controls:

Add the LOA5 walk cycles to the classroom. Put buttons on the chalkboard to switch modes.

Each cycle (Walk, Run, Jump) was extracted from the standardized animation as a `TimeSensor` plus 147 interpolators and routes; an ECMAScript `Script` node enables exactly one at a time, triggered by `TouchSensor` button panels on the board.

The final shape came from short, corrective prompts—the texture of real iteration:

Make the skeleton stop waving like that.

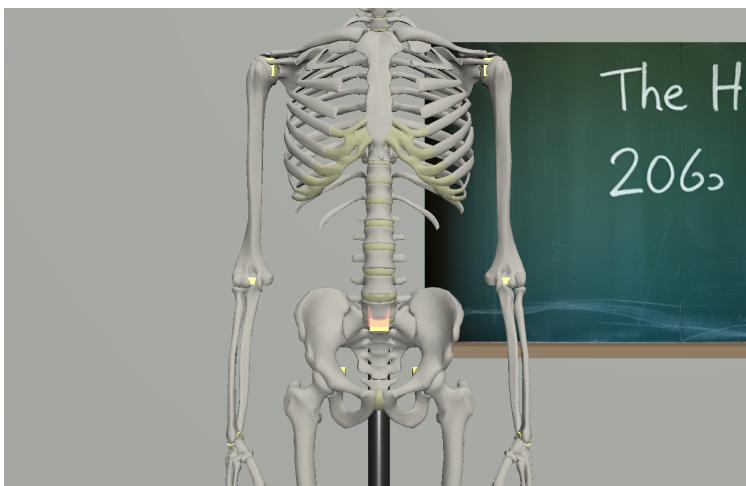


Figure 5: After the PBR pass: the ribcage, spine, and pelvis recover three dimensional form from `EnvironmentLight`, instead of the flat white wash the high-ambient Phong materials produced.

The buttons should be styled to blend in with the chalkboard.

Lesson: late-stage feedback is mostly aesthetic and behavioral, and it arrives in small increments. A fast verify-by-render loop (Section 6) is what lets each one-line note resolve in a single pass. These produced the rest pose (arm down) and the chalk-drawn buttons of Figure 6.

6 Practices That Made It Work

Verify by rendering, not by hoping.

The single most important habit: after every change, render the scene to an image and *look*. Castle Model Viewer’s command-line screenshot mode and a headless WebGL browser let the agent catch an invisible humanoid, a neon-glowing skull, a blank chalkboard, and washed-out lighting—each of which validated against the schema but looked wrong. Schema-valid is not the same as correct.

Integrate the canon; don’t reinvent it.

Adopting the official skeleton, package, and stylesheet meant the work composed with the broader ecosystem and surfaced genuine issues worth reporting upstream.

Separate the authoritative file from the rendered page.

The `.x3d` keeps real HAnim nodes for standards tools; the browser page is generated from it. When a player cannot render a construct, adapt the *page*, not the source of truth.

Build in stages, commit at each.

Each milestone (skeleton, LOA5, PBR, interactivity) was a separate version-control commit, so any step could be revisited without losing the others.

Treat tool friction as contribution.

Bugs found while using the tools became fixes and reports back to them, not workarounds buried in the demo.

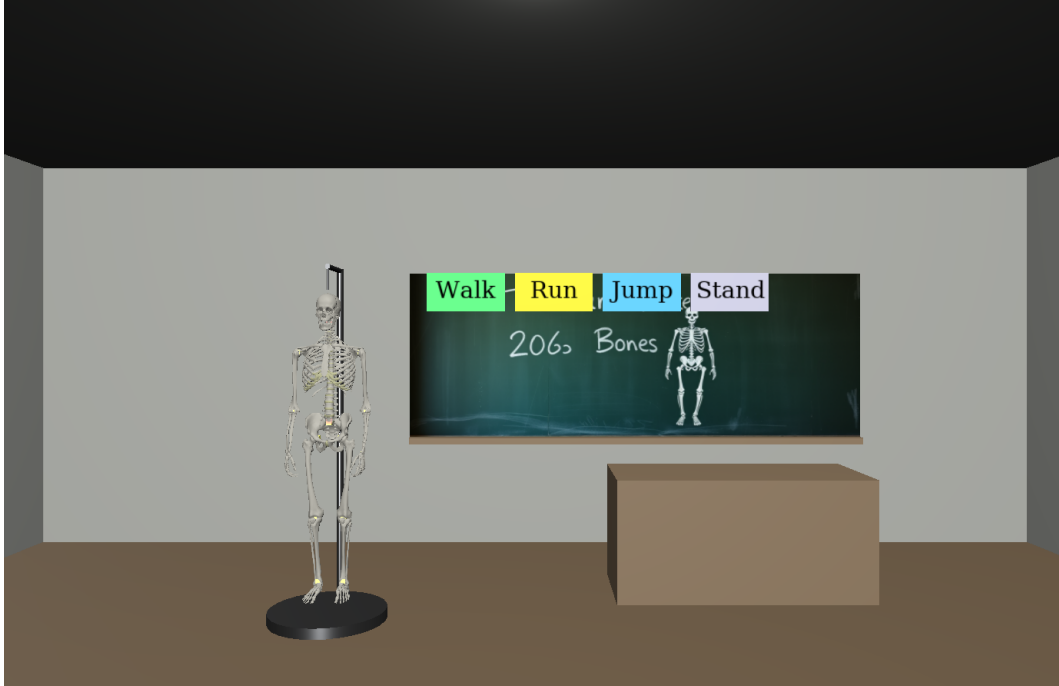


Figure 6: The classroom: the canonical LOA5 skeleton on a display stand with PBR bones, FLUX-generated chalkboard and posters, and clickable Walk/Run/Jump/Stand buttons that switch the standardized motion cycles.

7 Findings and Contributions

- **MCP fixes.** Using the renderer and validator on real HAnim content exposed defects: the X3DOM page renderer crashed on XML comments and failed to escape attribute quotes; the semantic checker rejected legal `set_/_changed` ROUTE aliases and flagged conformant USE references and empty HAnim joints. These were fixed and submitted upstream.
- **x3d.py serialization bugs.** The package omits `containerField` on nodes placed in a non-default field (e.g. a joint in the `skeleton` field, a texture in `baseTexture`), and defaults `EnvironmentLight.global` to `true` against the spec’s `false`. Both yield schema-valid XML that renders wrong. Documented with reproductions for the maintainers.
- **X3DOM HAnim gap.** Current X3DOM does not render an HAnim skeleton and trips a `MutationObserver` initialization error on large animated scenes. X_ITE renders the same content correctly. This is a concrete opportunity to contribute to X3DOM.
- **Anatomical-layer roadmap.** The skeleton is a frame to build outward from—clothing/coveroids, ligaments, muscles (via `HAnimDisplacer` morphs), organs, nerve/vascular networks, skin (weighted `HAnimHumanoid.skin`), and hair—each a candidate standards contribution.

8 Reproducing the Demo

1. **Environment.** Install a JDK, the x3d Python package, Saxon-HE (XSLT 2.0), and a viewer (Castle Model Viewer for desktop; a modern browser for X_ITE). Optionally install mflux for

texture generation.

2. **Generate.** Run the generator scripts; each emits an `.x3d` scene and its web page. The skeleton fragment and bone meshes live under `assets/loa5/`.
3. **Validate.** The scenes pass the X3D 4.0 schema and the MCP semantic checker (zero errors).
4. **View.** Open the `.x3d` in Castle Model Viewer (or X3D-Edit), or open the generated `.html` in a browser for X_ITE. In the classroom, click *Walk*, *Run*, *Jump*, or *Stand* on the chalkboard to switch the figure's motion.

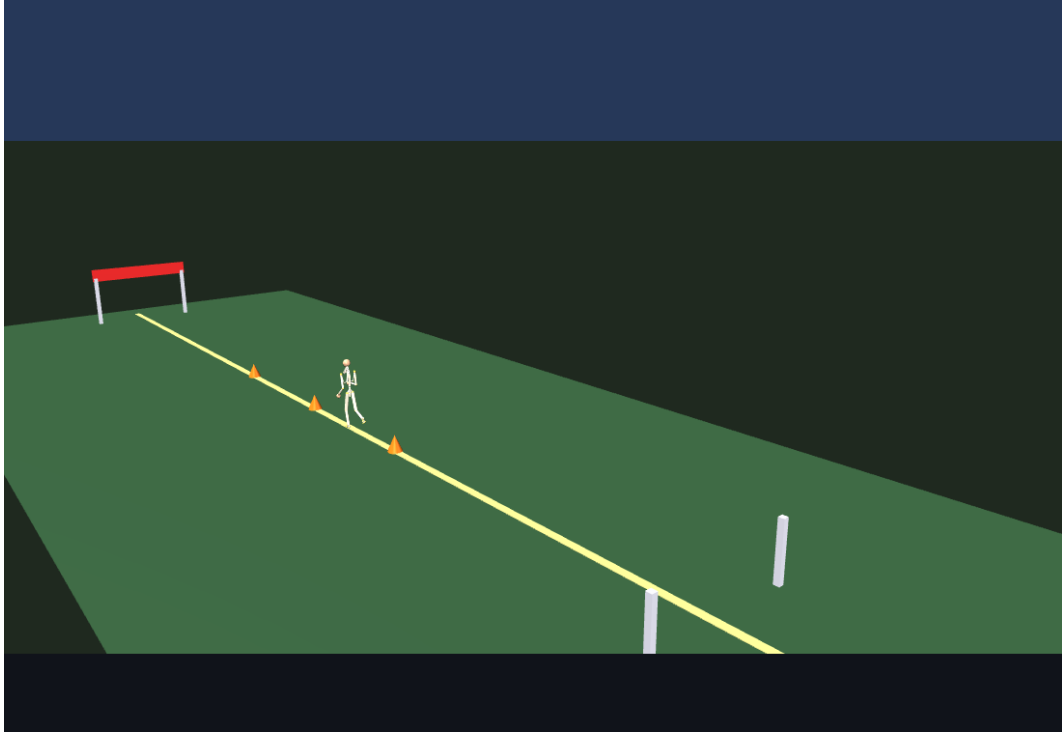


Figure 7: The runner demo delivered to the browser via X_ITE: the real HAnim figure traverses the slalom course with the standardized gait. X_ITE renders the HAnim skeleton directly, with no flattening.

9 Conclusion

A capable model and a person who knows the domain are stronger together than either alone, and saying so plainly—“we built this”—is both more accurate and more useful than the marketing shorthand. The concrete enablers were modest: expose the toolchain through an MCP, integrate the actual standard, and close a verify-by-render loop so the agent is accountable to what the scene looks like, not just whether it parses. The result is a standards-conformant, interactive LOA5 humanoid demonstration—and a short list of contributions handed back to the tools that made it.